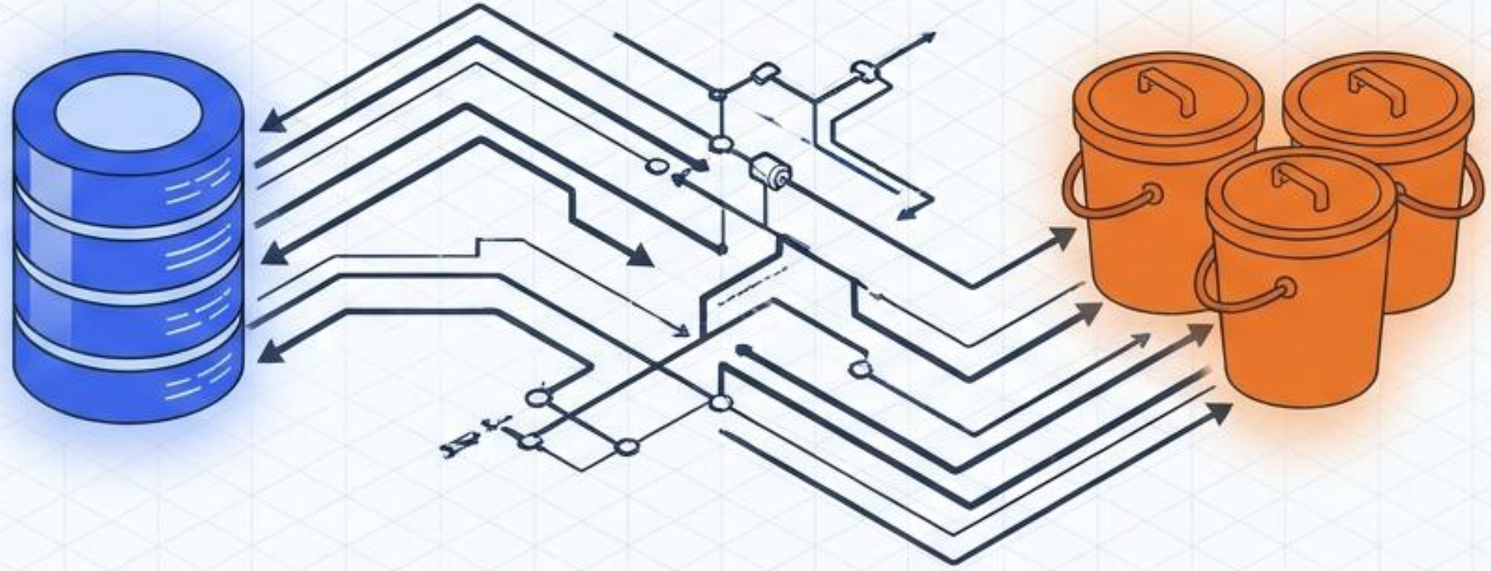
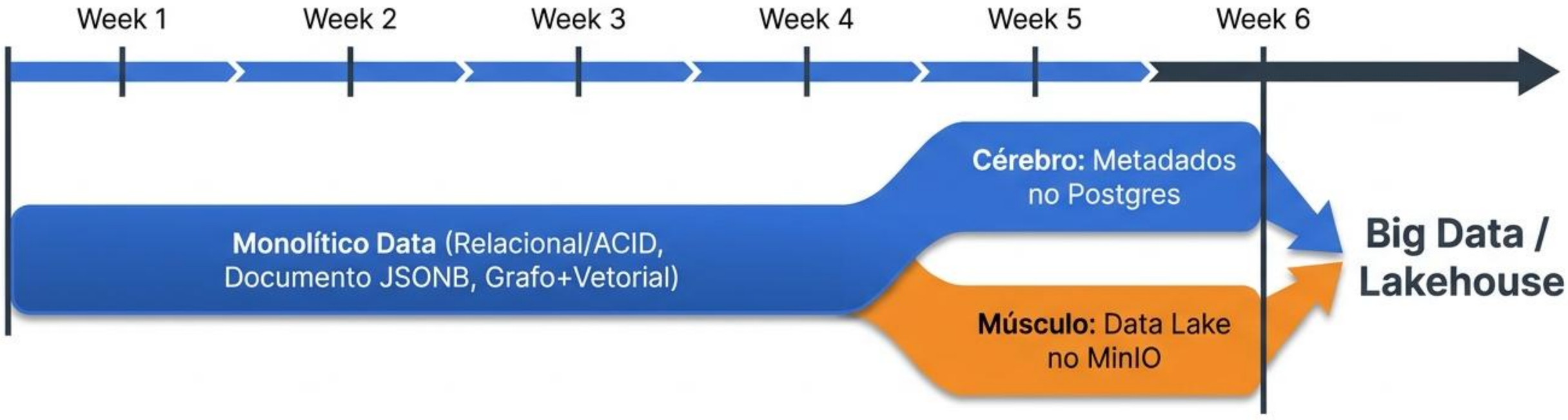
SwiftTrack IoT: O Paradigma de Objetos

Separando o Cérebro do Músculo na Arquitetura de Dados



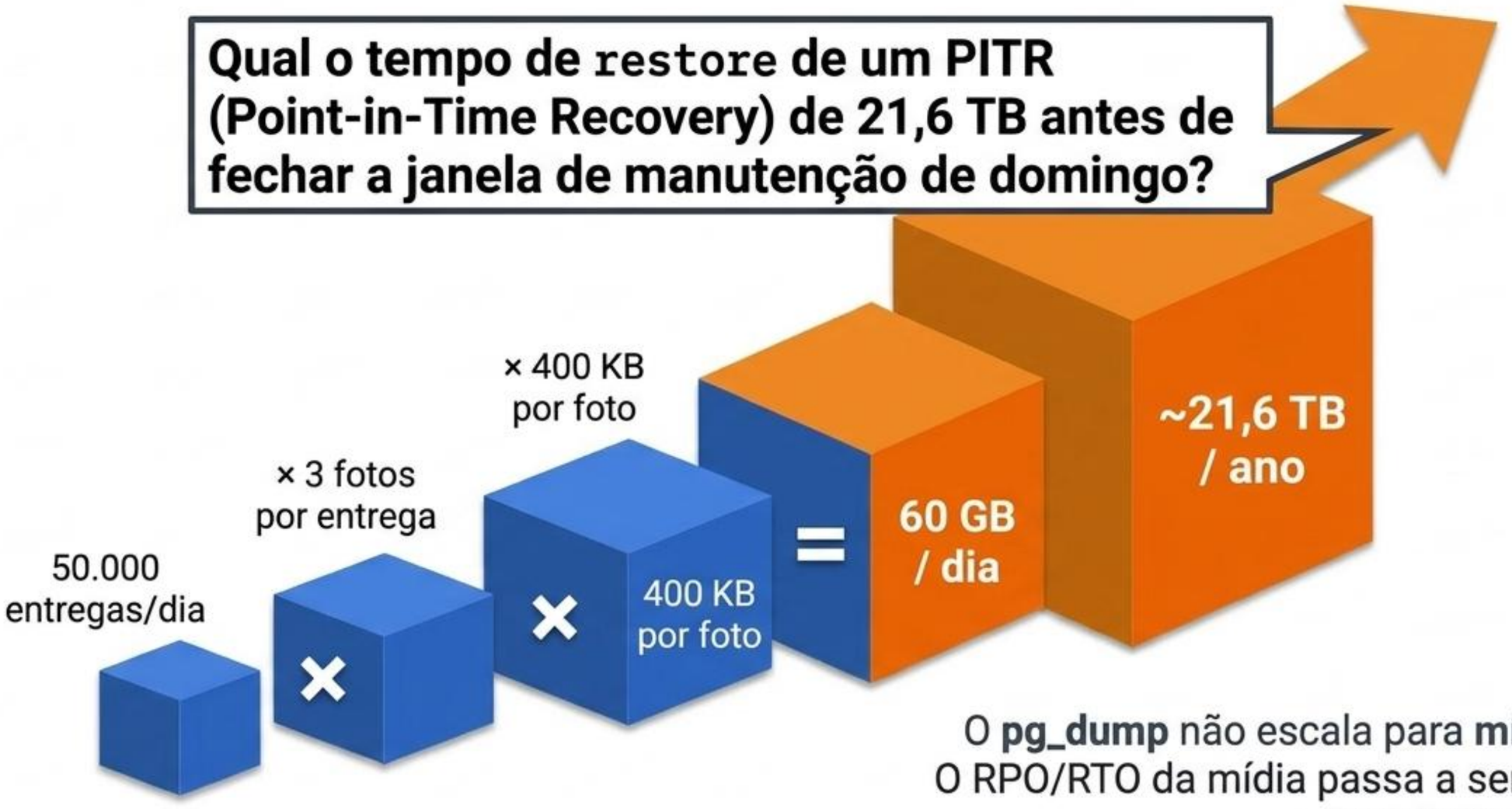
Posicionamento no Fio Condutor



A Semana 5 não introduz um segundo banco de dados. Ela introduz a separação entre plano de controle e plano de dados — o pré-requisito físico para o Lakehouse.

A Bomba-Relógio do Backup

Qual o tempo de `restore` de um PITR (Point-in-Time Recovery) de 21,6 TB antes de fechar a janela de manutenção de domingo?



O **pg_dump** não escala para **mídias contínuas**.
O RPO/RTO da mídia passa a ser indevidamente
o RPO/RTO do faturamento.

To BLOB or Not To BLOB?

	Postgres (BLOB)	MinIO / S3 (Objeto)
Amplificação de WAL (digrostica on úrila)	Cada byte vira WAL; réplicas inflam	Zero impacto relacional
TOAST I/O	Leituras fragmentadas e descompactação CPU-bound	Stream direto de disco/rede
Cache Pollution	Expulsa shared_buffers, degradando OLTP	Cache isolado/Edge
Custo de Storage	SSDs caros de alto IOPS	Discos capacitivos / WORM

A **Regra de Ouro** (Sears, van Ingen & Gray, Microsoft Research):

- **< 64 KB**: Mantém no banco (latência superior, atomicidade estrita).
- **> 256 KB**: Object Storage (sem discussão). (Foto do comprovante SwiftTrack = 400 KB).

O Novo Paradigma: Cérebro vs. Músculo

O Cérebro (Postgres)



Verdade transacional
ACID
Metadados
Mutável

O Músculo (MinIO / S3)



Bytes brutos
WORM (Write Once, Read Many)
Escalabilidade horizontal
Imutável

Compatibilidade de API como decisão de projeto: O mesmo código boto3 roda localmente no MinIO (Edge Hubs) e na nuvem (AWS S3).

Anatomia do Object Storage



Bucket + Chave (Não é uma pasta hierárquica, é um namespace plano: a/b/c.jpg)

Bytes (O conteúdo opaco)

```
{ content-type: image/jpeg
  last-modified: 2024-05-01T10:00:00Z
  x-amz-meta-delivery-id: 12345 }
```

Metadados (Sistêmicos e definidos pelo usuário, ex: delivery-id)



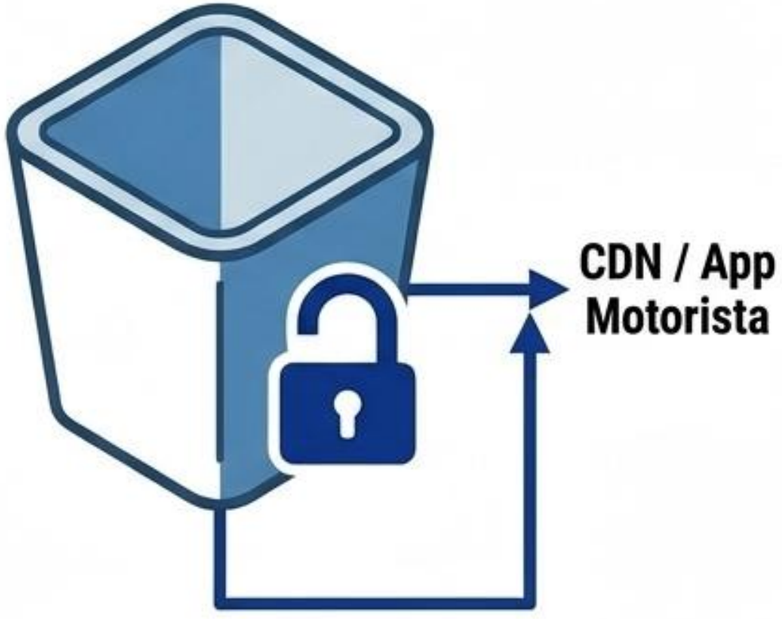
Versão (Não existe UPDATE parcial; sobrescrever gera nova versão)



Strong Consistency: O mito da "consistência eventual" no S3 acabou em dez/2020. Leituras após escritas são estritamente consistentes.

Estratégia de Buckets & Compliance (LGPD)

swifttrack-static



Público: Apenas estáticos (logos, JS).

swifttrack-documents



Chaveamento Temporal: Convenção pod/{ano}/{mes}/{dia}/{uuid}.jpg habilita retenção (ILM) e partition pruning.

swifttrack-proof-of-delivery



Privado (403): Restrito por padrão. Retornar status 200 público para comprovante com rosto = Vazamento LGPD.

O Contrato de Metadados

delivery_proof

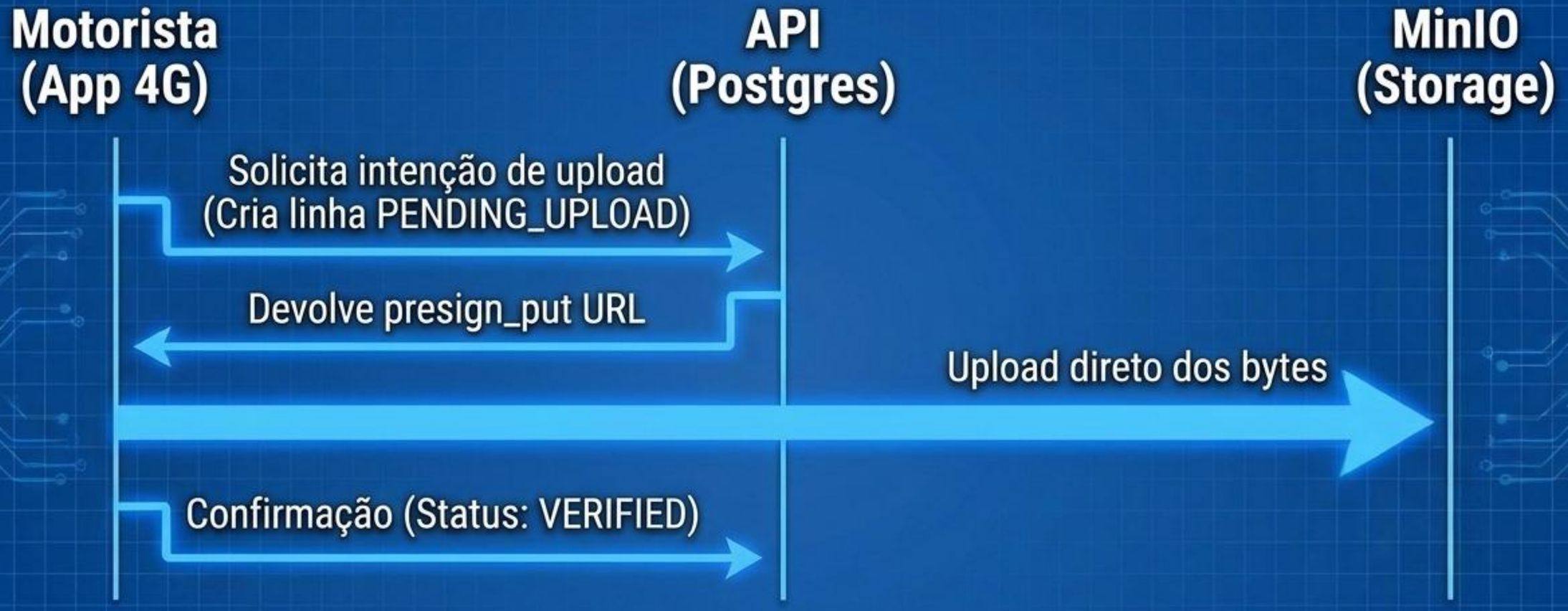
- proof_id (uuid)
- delivery_id (uuid)
- status (enum)
- **bucket** (text)
- **object_key** (text)
- **sha256** (char64)

Contrato de Metadados



Como não existe Two-Phase Commit (2PC) ou Chave Estrangeira entre Postgres e S3, a integridade do sistema depende inteiramente deste contrato. O banco valida estado, tamanho, e checksum (SHA256) antes de considerar a mídia válida.

A Dança da Integração: URLs Pré-Assinadas



Por que usar **presign_put**? Salva os workers da API de atuarem como "proxies de bytes" para conexões 4G lentas, garantindo que o plano de controle **permaneça leve e rápido**.

O Problema dos Órfãos e o Reaper

Passos 1 e 2 funcionam, mas App morre antes do passo 3.



Falha de Rede

Passo 3 funciona, mas App morre antes do passo 4.

Órfão no Banco

Linha PENDING_UPLOAD no Postgres sem objeto físico.

Órfão no Storage

Objeto no bucket, mas Postgres não sabe (pagando storage eterno).



Solução: Consistência Eventual com Reconciliação (The Reaper)

Um script Python varre o Postgres a cada 24 horas buscando status PENDING_UPLOAD. Faz um **HEAD Object** no MinIO.
Se 404 -> **DELETE** no banco. Se 200 -> Confere **SHA256** e marca **VERIFIED**.

Plano de Entrega e a Armadilha do CDN

URL com **?X-Amz-Signature=...**



Hit rate = 0%. Query strings dinâmicas alteram a cache key a cada requisição, destruindo o cache.

CDN
(Nginx/CloudFront)

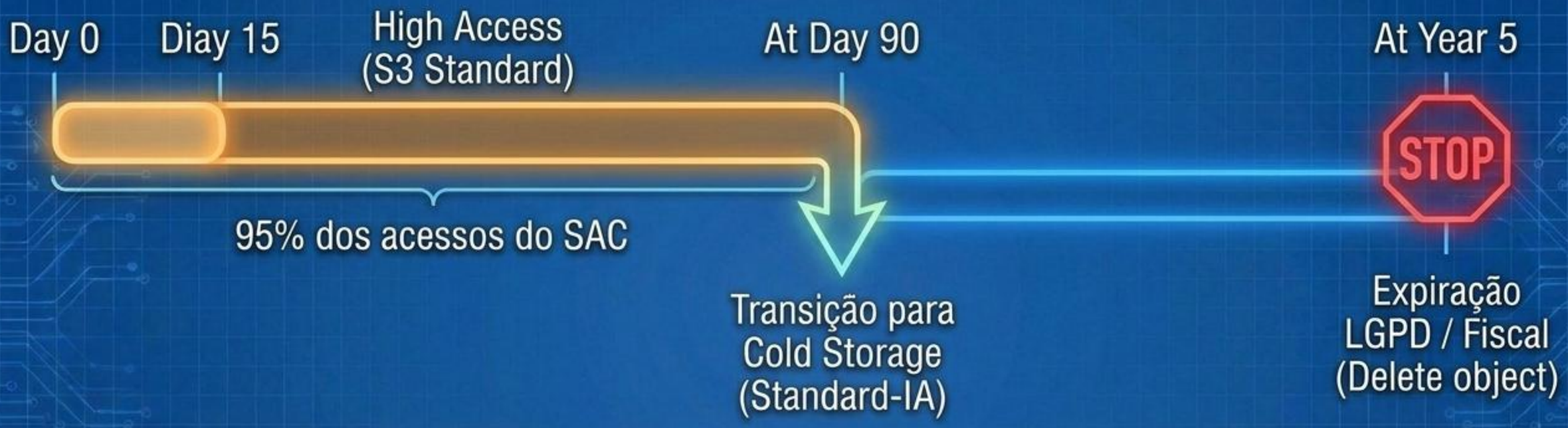
Requisição direta sem query string



A assinatura deve ocorrer na borda (Edge via CloudFront OAC / Signed Cookies), não na origem.

CDN
(Nginx/CloudFront)

Ciclo de Vida: A Regra dos 90 Dias



Compliance (LGPD)

Comprovantes possuem valor probatório.
Expirar sumariamente em 90 dias é imprudente.
O correto é Transição, com expiração em 5 anos.

Nota Técnica

O gatilho de transição é o metadado Last-Modified, que é sistêmico e impossível de falsificar via API.

Economia de Escala (Custos)

Tiers de Armazenamento	Custo Unitário (por GB)	Estimativa Mensal (21.6 TB)
S3 Standard	US$ 0,023/GB	~US$ 509/mês
S3 Standard-IA	US$ 0,0125/GB	~US$ 276/mês
Glacier Deep Archive	US$ 0,00099/GB	~US$ 22/mês

Por que não jogar tudo direto no Glacier?

O design precisa considerar o Custo de Recuperação e a latência.
Quando o SAC precisa da foto de entrega de ontem em tempo real,
os milissegundos do Standard-IA justificam o custo em relação ao Deep Archive.

Registro de Decisão Arquitetural

Título: ADR-05: Storage de Mídias via Object Storage

Contexto: Crescimento de mídias de entrega (21,6 TB/ano) ameaçando SLA do banco relacional.

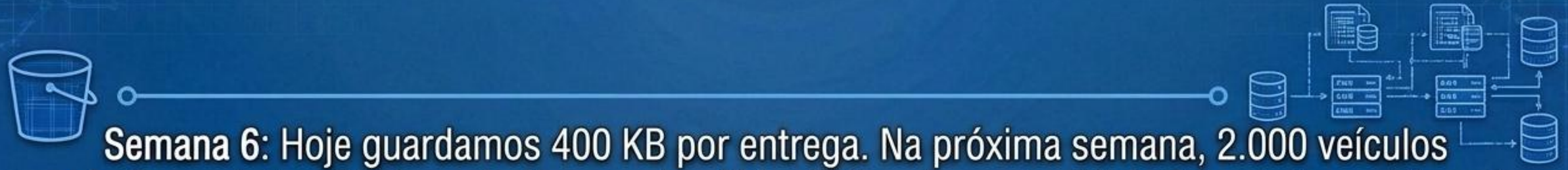
Decisão: Adoção de API compatível com S3 (MinIO) integrada via Contrato de Metadados no Postgres.

Alternativas Rejeitadas: Postgres BLOB/bytea (rejeitado por WAL amplification e cache pollution).

Consequências: Exige implementação de rotina de Reaper para consistência eventual de órfãos, e gerenciamento de expiração de URLs pré-assinadas.

Conclusão & Próximos Passos

> O Postgres guarda a verdade. O MinIO guarda o comprovante. A arquitetura vive na disciplina de manter os dois de acordo — porque nenhum protocolo fará isso por você.



Semana 6: Hoje guardamos 400 KB por entrega. Na próxima semana, 2.000 veículos enviarão posições GPS a cada 5 segundos (34 milhões de pontos/dia). O bucket provisionado hoje se tornará a fundação física do Lakehouse de amanhã. Objeto por ponto será viável?